Us app: Backend Endpoints

Project: couple_app (com.nammaempire.together) · Firebase project: together-48033 · Region: asia-south1 · Analysed 24 Oct 2026

1. Summary

The app has no traditional REST API. It talks to Firebase directly: most reads and writes go straight to Firestore and Storage (protected by security rules), and only the sensitive work goes through Cloud Functions. So "endpoints" here means four kinds of entry point:

Kind	Count	Technology	Where it's defined
Cloud Functions (server code)	5	Node.js 22, TypeScript, Firebase Functions v2	`functions/src/index.ts`
· Callable HTTPS (app calls them)	3	onCall	finalizeSeparation, leaveNow, verifyPurchase
· Scheduled job	1	onSchedule (Cloud Scheduler)	processSeparations
· Pub/Sub trigger	1	onMessagePublished	playBillingNotifications
Firestore data paths	36	Cloud Firestore (NoSQL)	`firestore.rules`
Storage file paths	11	Firebase Storage	`storage.rules`
External APIs used	4 services	Google Play API, oEmbed, Agora	functions/src/play.ts, lib/

Total: **5 server functions** (3 callable by the app), **36 Firestore paths**, **11 Storage paths**, and **4 outside services**. Everything not listed is denied by a catch-all rule.

2. Endpoint list (URLs)

Project **together-48033**, region **asia-south1**, bucket **together-48033.firebasestorage.app**. Every app endpoint needs a signed-in user: send the Firebase ID token as `Authorization: Bearer <ID_TOKEN>`. The app itself goes through the Firebase SDKs, which call these same endpoints for it.

2.1 Cloud Function endpoints (callable by the app): 3

Callable functions take `POST` with `Content-Type: application/json`, the input wrapped as `{"data": {...}}`, and answer `{"result": {...}}` (or `{"error": {...}}`).

#	Method	URL	Request body
1	POST	`https://asia-south1-together-48033.cloudfunctions.net/finalizeSeparation`	`{"data":{"coupleId":"..."}}`
2	POST	`https://asia-south1-together-48033.cloudfunctions.net/leaveNow`	`{"data":{"coupleId":"...","confirm":"LEAVE"}}`
3	POST	`https://asia-south1-together-48033.cloudfunctions.net/verifyPurchase`	`{"data":{"coupleId":"...","productId":"...","purchaseToken":"...","packageName":"com.nammaempire.together","gift":{"message":"..."}}}`

2.2 Background functions (no public URL): 2

#	Function	Triggered by
4	processSeparations	Cloud Scheduler job, every 15 minutes (Asia/Kolkata)
5	playBillingNotifications	Pub/Sub topic `projects/together-48033/topics/play-billing` (Google Play sends to it)

2.3 Firestore endpoints (36 paths)

Base URL: `https://firestore.googleapis.com/v1/projects/together-48033/databases/(default)/documents`. Add any path from section 4 to the end, for example `.../documents/couples/{coupleId}/messages/{messageId}`.

Method	URL pattern	Does
GET	`{base}/{collection}/{docId}`	Read one document
GET	`{base}/{collection}`	List a collection
POST	`{base}/{collection}`	Create a document (auto id)
PATCH	`{base}/{collection}/{docId}`	Create or update a document
DELETE	`{base}/{collection}/{docId}`	Delete a document
POST	`{base}:runQuery`	Query (where / orderBy / limit)

2.4 Storage endpoints (11 paths)

Base URL: `https://firebasestorage.googleapis.com/v0/b/together-48033.firebasestorage.app/o`. The file path (section 5) is URL-encoded, so `/` becomes `%2F`.

Method	URL pattern	Does
POST	`{base}?name={path}`	Upload a file
GET	`{base}/{path}?alt=media`	Download a file
GET	`{base}/{path}`	File details (size, type)

Method	URL pattern	Does
DELETE	`{base}/{path}`	Delete a file

Example: `https://firebasestorage.googleapis.com/v0/b/together-48033.firebasestorage.app/o/couples%2F{coupleId}%2Fphotos%2F{photoId}?alt=media`

2.5 Outside endpoints the code calls

Method	URL	Called by
GET	`https://androidpublisher.googleapis.com/androidpublisher/v3/applications/com.nammaempire.together/purchases/products/{productId}/tokens/{token}`	verifyPurchase (server)
POST	`https://androidpublisher.googleapis.com/androidpublisher/v3/applications/com.nammaempire.together/purchases/products/{productId}/tokens/{token}:acknowledge`	verifyPurchase (server)
GET	`https://androidpublisher.googleapis.com/androidpublisher/v3/applications/com.nammaempire.together/purchases/subscriptionsv2/tokens/{token}`	verifyPurchase, playBillingNotifications
POST	`https://androidpublisher.googleapis.com/androidpublisher/v3/applications/com.nammaempire.together/purchases/subscriptions/{subscriptionId}/tokens/{token}:acknowledge`	verifyPurchase (server)
GET	`https://www.youtube.com/oembed?format=json&url={link}`	Music (app)
GET	`https://open.spotify.com/oembed?url={link}`	Music (app)
GET	`https://soundcloud.com/oembed?format=json&url={link}`	Music (app)
SDK	Agora RTC (App ID, no URL of yours)	Calls (app)

3. Cloud Functions

All five are 2nd-gen functions in region asia-south1. Callables require a signed-in user; the app calls them with the cloud_functions Flutter package.

3.1 finalizeSeparation

Endpoint	`POST https://asia-south1-together-48033.cloudfunctions.net/finalizeSeparation`
Type	Callable HTTPS (onCall)
Called by	App, `lib/features/couple/data/end_pairing_gateway.dart`, when it opens after the separation date
Input	`{ coupleId: string }`
Returns	`{ status: 'ended' | 'alreadyEnded' | 'inProgress', how: string | null }`
What it does	Checks the caller is a member and the separation date has passed, then runs endCouple(): deletes all the couple's files and subcollections, invites and pair requests, unpairs both users (their accounts and profiles stay), and deletes the couple doc last.
Errors	unauthenticated, invalid-argument, permission-denied, failed-precondition (no separation, or not yet due)
Config	timeout 300 s, 512 MiB

3.2 leaveNow

Endpoint	`POST https://asia-south1-together-48033.cloudfunctions.net/leaveNow`
Type	Callable HTTPS (onCall)
Called by	App, `end_pairing_gateway.dart` (gated behind app lock / biometrics on the phone)
Input	`{ coupleId: string, confirm: 'LEAVE' }`
Returns	`{ status, how }` (same shape as above, how = 'leftNow')
What it does	Ends the pairing immediately, whether or not a separation was requested. Same cleanup as above.
Errors	unauthenticated, invalid-argument (bad coupleId or confirm isn't 'LEAVE'), permission-denied
Config	timeout 300 s, 512 MiB

3.3 verifyPurchase

Endpoint	`POST https://asia-south1-together-48033.cloudfunctions.net/verifyPurchase`
Type	Callable HTTPS (onCall)
Called by	App, `lib/features/premium/data/purchase_verifier.dart`, after every Google Play purchase and on startup as a restore
Input	`{ coupleId, productId, purchaseToken, packageName, gift?: { message: up to 300 chars } }`
Returns	`{ status: 'granted' | 'alreadyGranted', premiumUntil?, bookCredits?, giftId? }`
What it does	Verifies the purchase with the Google Play Developer API, grants it to the couple exactly once per token (records it in purchases/{sha256(token)}), then acknowledges it. Products: premium_monthly, premium_yearly, book_single, gift_premium_1m / 3m / 12m, gift_book.
Errors	unauthenticated, invalid-argument, permission-denied, failed-precondition (pending / cancelled / wrong product / couple ending), unavailable (Play API error, retry later)
Config	timeout 60 s

3.4 processSeparations

Endpoint	No public URL (Cloud Scheduler)
Type	Scheduled (Cloud Scheduler), every 15 minutes, Asia/Kolkata
Called by	Google Cloud Scheduler (not the app)
Input	None
Returns	None (logs a tally)
What it does	Finds up to 100 couples whose separation.effectiveAt has passed and ends each one with endCouple(). Anything left over is picked up on the next tick.
Errors	Failures are logged per couple; no retries (next run retries)
Config	timeout 540 s, 512 MiB, retryCount 0

3.5 playBillingNotifications

Endpoint	No public URL (Pub/Sub topic play-billing)
Type	Pub/Sub trigger, topic `play-billing`
Called by	Google Play Real-time Developer Notifications (not the app)
Input	Play DeveloperNotification JSON
Returns	None
What it does	Keeps premium in sync: on subscription events it re-fetches the subscription and recomputes premium.until; on voided purchases it removes an unused book credit and marks the purchase voided.
Errors	Transient errors are retried by Pub/Sub for up to 24 hours, then dropped
Config	timeout 60 s, retry on

4. Firestore data paths (36)

The app reads and writes these directly through the cloud_firestore package. Every path is guarded by `firestore.rules` (about 1,100 lines); couple data is readable only by the two members. "Ops" shows what the rules allow from the app (each still has conditions).

#	Path	Ops	Used for
1	`users/{userId}`	read, create, update, delete	User profile, current coupleId, chat look settings
2	`users/{userId}/devices/{deviceId}`	read, create, update, delete	Signed-in phones (device trust)
3	`users/{userId}/securityEvents/{eventId}`	read, create, update, delete	Security log (new sign-ins, phone changes)
4	`phoneIndex/{e164}`	get, list, create, update, delete	Phone number to user lookup for pairing
5	`pairRequests/{requestId}`	read, create, update, delete	Requests to pair by phone number
6	`invites/{code}`	get, list, create, update, delete	Invite codes (one at a time, can't be enumerated)
7	`purchases/{purchaseId}`	none (server only)	Purchase records, written only by verifyPurchase
8	`couples/{coupleId}`	read, create, update, delete	The couple: members, start date, premium, separation
9	`couples/{coupleId}/messages/{messageId}`	read, create, update, delete	Chat messages, reactions, pins, call logs
10	`couples/{coupleId}/moods/{userId}`	read, write	Each partner's current mood
11	`couples/{coupleId}/dailyQuestions/{dateKey}`	read, create, update, delete	Daily question and both answers
12	`couples/{coupleId}/call/{callId}`	read, create, update, delete	Call signalling (ringing / accepted / ended)
13	`couples/{coupleId}/memories/{memoryId}`	read, create, update, delete	Memories and albums
14	`couples/{coupleId}/events/{eventId}`	read, create, update, delete	Shared calendar events and important dates
15	`couples/{coupleId}/notes/{noteId}`	read, create, update, delete	Love notes
16	`couples/{coupleId}/notes/{noteId}/sealed/{docId}`	get, list, create, update, delete	Sealed note content until it can be opened
17	`couples/{coupleId}/datePlans/{planId}`	read, create, update, delete	Date planner
18	`couples/{coupleId}/bucketList/{itemId}`	read, create, update, delete	Bucket list
19	`couples/{coupleId}/wishlist/{itemId}`	read, create, update, delete	Wishlist items
20	`couples/{coupleId}/wishlistClaims/{itemId}`	read, create, update, delete	Who has claimed a wishlist gift
21	`couples/{coupleId}/places/{placeId}`	read, create, update, delete	Saved places

#	Path	Ops	Used for
22	`couples/{coupleId}/expenses/{expenseId}`	read, create, update, delete	Shared expenses
23	`couples/{coupleId}/deviceChecks/{checkId}`	read, create, update, delete	Partner approval for a new device
24	`couples/{coupleId}/gifts/{giftId}`	read, create, update, delete	Gifts (premium / book) to reveal
25	`couples/{coupleId}/books/{bookId}`	read, create, update, delete	Memory books
26	`couples/{coupleId}/vault/{docId}`	read, create, update, delete	Vault setup (encrypted keys, passcode state)
27	`couples/{coupleId}/vaultItems/{itemId}`	read, create, update, delete	Encrypted vault items
28	`couples/{coupleId}/vaultEvents/{eventId}`	read, create, update, delete	Vault activity log
29	`couples/{coupleId}/songs/{songId}`	read, create, update, delete	Shared songs
30	`couples/{coupleId}/videos/{videoId}`	read, create, update, delete	Shared videos
31	`couples/{coupleId}/listening/{docId}`	read, create, update, delete	Listen-together sync state
32	`couples/{coupleId}/watching/{docId}`	read, create, update, delete	Watch-together sync state
33	`couples/{coupleId}/settings/{docId}`	read, create, update, delete	Couple-wide settings
34	`couples/{coupleId}/games/{gameId}`	read, create, update, delete	Live games (Ludo, Truth or Dare, etc.)
35	`couples/{coupleId}/games/{gameId}/secret/{docId}`	get, list, create, update, delete	Hidden game answers (Know Me)
36	`couples/{coupleId}/nudges/{nudgeId}`	read, create, update, delete	Love taps / nudges

"Used for" is inferred from path names and the matching feature folders in lib/features.

5. Storage file paths (11)

Files are uploaded and downloaded directly with the firebase_storage package, guarded by `storage.rules`. Any other path is denied.

#	Path	Ops	Holds
1	`couples/{coupleId}/photos/{photoId}`	read, write, delete	Chat photos
2	`couples/{coupleId}/videos/{videoId}`	read, write, delete	Chat videos
3	`couples/{coupleId}/voice/{voiceId}`	read, write, delete	Voice messages
4	`couples/{coupleId}/memories/{fileId}`	read, create, update, delete	Memory photos and videos
5	`couples/{coupleId}/notes/{fileId}`	read, create, delete	Love-note attachments
6	`couples/{coupleId}/profile/{photoId}`	read, write, delete	Couple profile photo
7	`couples/{coupleId}/backgrounds/{fileId}`	read, create, delete	Shared chat backgrounds

#	Path	Ops	Holds
8	`couples/{coupleId}/books/{fileId}`	read, create, delete	Memory book PDFs and images
9	`couples/{coupleId}/vault/{fileId}`	read, create, delete	Encrypted vault files
10	`users/{userId}/backgrounds/{fileId}`	read, create, delete	Personal chat backgrounds
11	`users/{userId}/profile/{photoId}`	read, write, delete	User profile photo

6. External services called

Service	Called from	Endpoints	Purpose
Google Play Developer API (androidpublisher v3)	Server: `functions/src/play.ts`	purchases.products.get, purchases.subscriptionsv2.get, purchases.products.acknowledge, purchases.subscriptions.acknowledge	Verify and acknowledge purchases
oEmbed (no API key)	App: `song_metadata_service.dart`	`youtube.com/oembed`, `open.spotify.com/oembed`, `soundcloud.com/oembed`	Song title and artwork from a pasted link
Agora RTC	App: `lib/features/calls/`	Agora SDK (App ID only)	Voice and video calls
Firebase Authentication	App	Email, Google, phone sign-in	Sign-in (managed by Firebase)

7. Notes

- **Only 3 endpoints are called by the app as functions.** Everything else the app does is direct Firestore/Storage access, so the security rules are effectively your API; they're covered by 13 rules test suites in firestore-tests/.
- **Agora is in "Testing mode: APP ID", with no token server.** For production you'd add a token endpoint, most naturally a fourth callable function (e.g. getAgoraToken) next to the existing ones.
- **A beforeSignIn blocking function is mentioned but not built.** device_trust_controller.dart refers to one as a possible future step; it isn't in functions/src.
- **Deploying needs the Blaze plan** (Cloud Scheduler and Pub/Sub). Expected cost at current scale is within free tiers.